

Zombie Apocalypse Survival Game

CS2001: Data Structures Semester Project

Your Name
Your Partner's Name (if applicable)
Section: Your Section

Spring 2025

1 Introduction

This report documents the design and implementation of the **Zombie Apocalypse Survival Game**, a console-based application developed for the CS2001 Data Structures course. The game challenges players to navigate a zombie-infested city to reach a safe zone while collecting items and avoiding zombies.

The project demonstrates practical applications of key data structures:

- Graph for city map representation and navigation
- Queue for zombie wave management
- Singly linked list for inventory tracking

2 Game Design and Rules

2.1 Game Overview

The game features:

- Two gameplay modes: Manual and Automated
- ASCII-based text user interface (TUI)
- Randomly generated maps with walls, items, and zombies
- Scoring system based on collected items and survival time

2.2 Game Rules

- The player (**P**) starts at position (1,1)
- The safe zone (**S**) is located at (width-1, height-1)
- Zombies (**Z**) move toward the player each turn
- Items (*****) provide health and score bonuses when collected
- Walls (**#**) are impassable obstacles
- Game ends when:
 - Player reaches safe zone (win)
 - Zombie catches player (lose)
 - Player health drops to 0 (lose)

3 Data Structures Implementation

3.1 Graph for City Map

The game map is implemented as a graph where:

- Each cell is a node
- Edges connect adjacent cells (up, down, left, right)
- Node types store cell content (wall, item, zombie, etc.)
- Used for pathfinding in automated mode

Listing 1: Graph Initialization

```
Graph::Graph(int vertices) {  
    numVertices = vertices;  
    adjacencyList.resize(vertices);  
    nodeTypes.resize(vertices, '-');  
    nodeColors.resize(vertices, 0);  
}
```

3.2 Breadth-First Search (BFS)

The automated mode uses BFS for pathfinding:

Listing 2: BFS Implementation

```
Position* Game::findShortestPath(int startX, int startY,  
                                int endX, int endY, int& pathLength) {  
    // BFS implementation to find shortest path  
    // Returns array of Positions representing the path  
}
```

3.3 Queue for Zombie Management

A custom queue manages zombie spawning and movement:

Listing 3: Zombie Queue

```
template <typename T>  
class SimpleQueue {  
    // Queue implementation with push, pop, peek operations  
    // Used to process zombies each turn  
};
```

3.4 Linked List for Inventory

A singly linked list tracks collected items:

Listing 4: Inventory Management

```
class LinkedList {  
    // Stores item name, value, and color  
    // Supports insert, remove, and display operations  
};
```

4 User Interface

4.1 Main Menu

The game features a color-coded menu system:

- Start Manual Mode
- Start Automated Mode
- View High Scores
- Exit Game

4.2 Game Display

The game screen shows:

- Current map with player, zombies, items, and walls
- Health status (color changes when low)
- Current score
- Inventory list

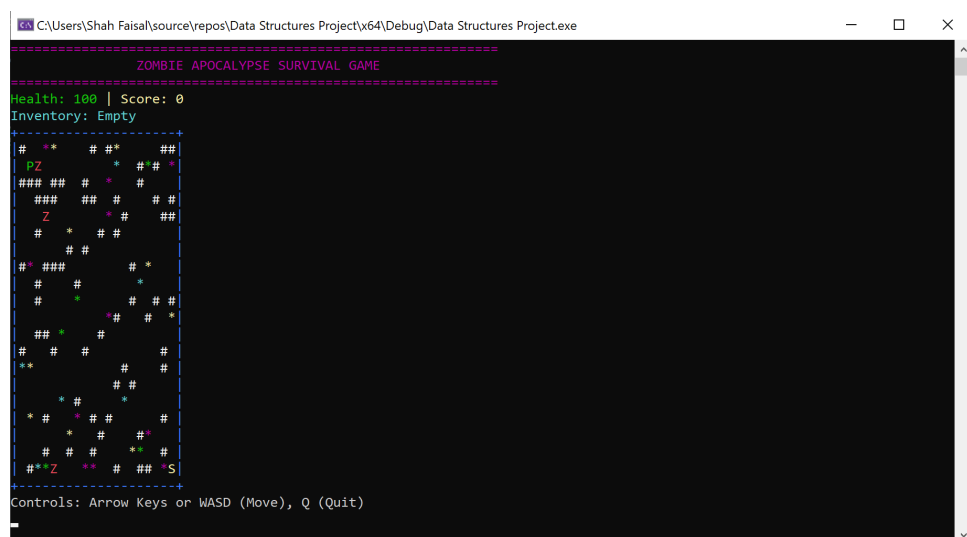


Figure 1: Game Screenshot Showing Map and Status Information

5 Scoring System

The scoring formula is:

$$Score = (ItemsCollectedValue \times 10) - (ZombiesEncountered \times 5) + (DistanceTraveled \times 2)$$

High scores are saved to `highscores.txt` with player names.

6 Testing and Challenges

6.1 Testing Approach

- Manual testing of all movement directions

- Verification of pathfinding algorithm
- Boundary condition testing (map edges, wall collisions)
- Inventory management tests

6.2 Challenges Faced

- Implementing efficient pathfinding
- Synchronizing zombie movements
- Managing console colors across platforms
- Balancing game difficulty

7 Conclusion

The Zombie Apocalypse Survival Game successfully demonstrates practical applications of graphs, queues, and linked lists in game development. The project met all requirements while providing an engaging gameplay experience.

Possible future enhancements include:

- Different zombie types with varied behaviors
- More complex maps with special zones
- Save/load game functionality
- Multiplayer support

Acknowledgments

We would like to thank our course instructor for guidance on data structures implementation and game design principles.

8 Game Snapshots

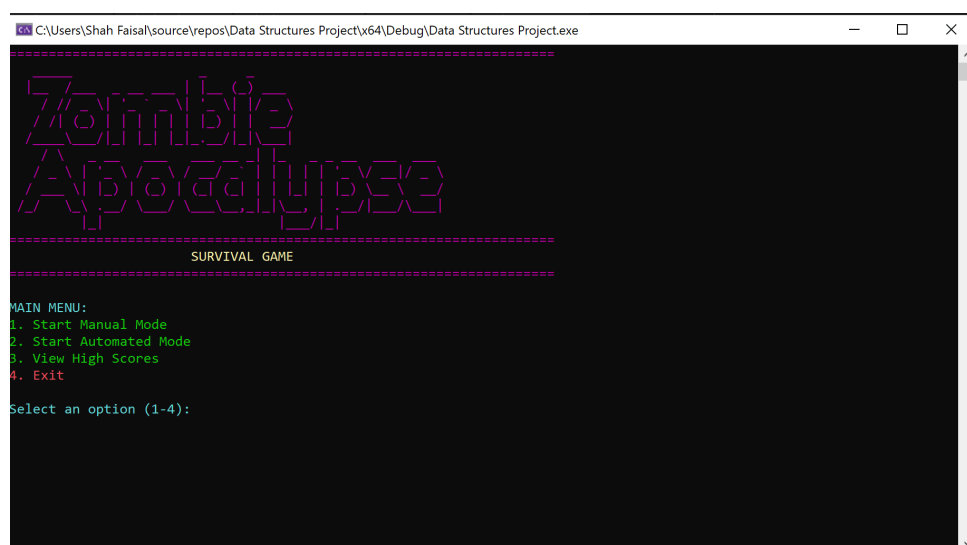


Figure 2: Game Start Screen Showing Main Menu Options

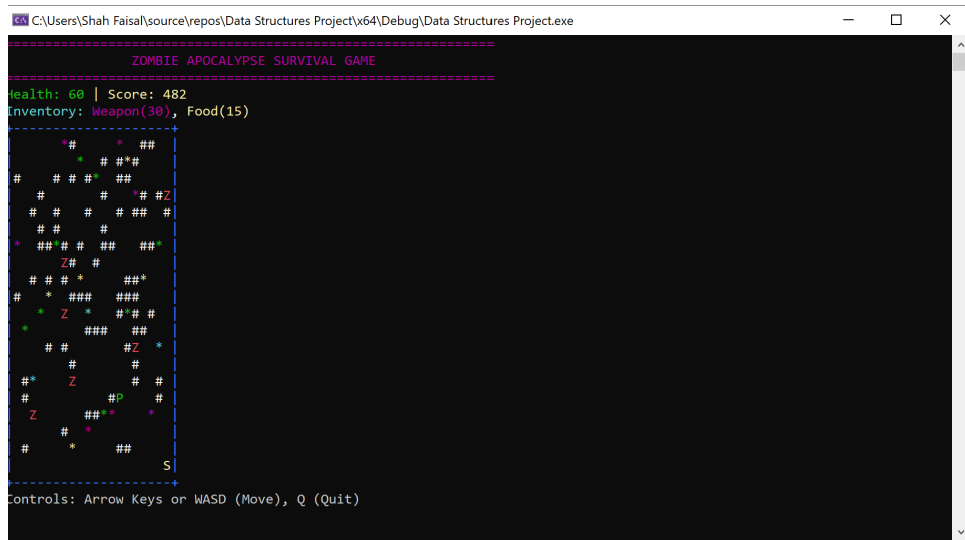


Figure 3: Gameplay In Progress with Player, Zombies, and Items Visible

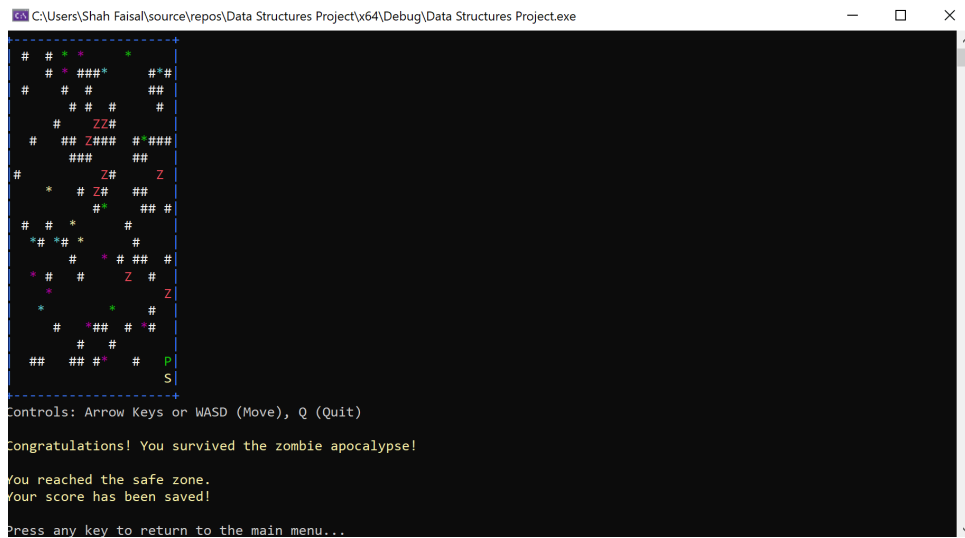


Figure 4: Winning Screen After Reaching Safe Zone

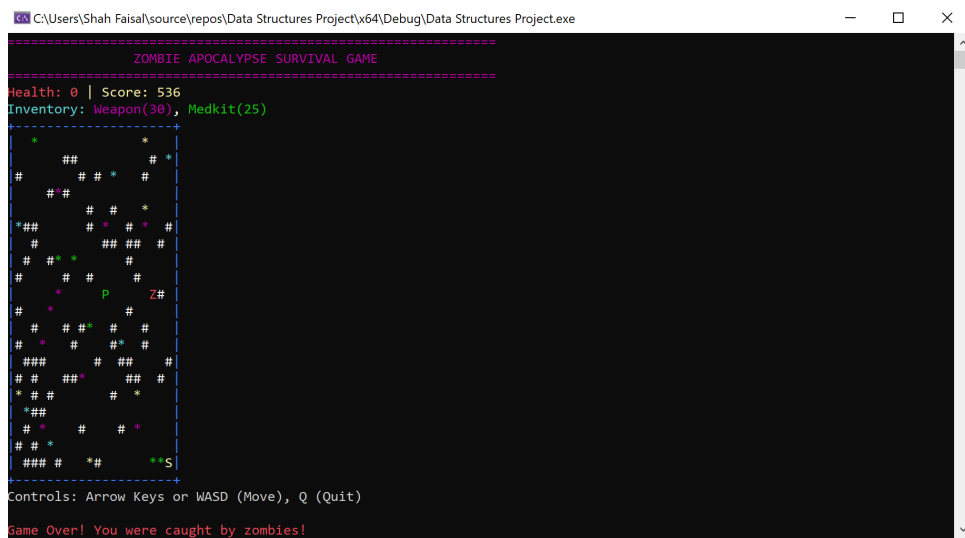


Figure 5: Losing Screen When Player is Caught by a Zombie or Health Reaches Zero

```
C:\Users\Shah Faisal\source\repos\Data Structures Project\Debug\Data Structures Project.exe
=====
HIGH SCORES
=====
Rank  Name      Score
-----
1     asim      1166
2     asim      536
3     faisal    1063
4     sir        470
5     Haseeb    291
6     asim      280
7     salman    695
8     shah      1065
9     uMAR      970
10    SHAH      477
11    Shah      802
12    shah      490
=====
Press any key to return to the main menu...
Press any key to return to the main menu...
```

Figure 6: High Scores Displayed After Game Ends